synchronized

要讲明白synchronized, 你首先得从计算机的发展史说起.

- CPU速度非常快, 但是内存速度比较慢. 为了缓解CPU和内存速度不一致的问题, 就有了三级缓存.
- 一级和二级缓存是CPU核心私有的. 第三级缓存是共享的.
- 那么问题来了在多线程的环境下, 当某一个CPU核心去执行某个线程任务, 把内存中的数据读到缓存中, 然后又修改了缓存中的数据. 当缓存数据还没有同步到主存时, 那线程B, 从主存中读数据那读的不就是旧数据了吗.
- 一个线程修改了共享数据, 但是另一线程无法立刻获取到最新的共享数据, 这就是可见性问题. 一说到这儿很多就能想到volatile是专门解决这个问题的.
- 除此之外, 执行代码的时候, CPU或编译器可能对指令重排序，导致执行顺序与代码顺序不一致. 这就是有序性问题.
- 除了可见性, 有序性问题, 还有一个原子性问题, 这个就不用多介绍了.

synchronized是什么, 能解决什么问题?

- synchronized就是个锁, 能够保证某一段代码同一时间只能被拿到锁的线程访问. 这就是原子性.
- `synchronized` 同步语句块的实现使用的是 `monitorenter` 和 `monitorexit` 指令. monitorenter就是加锁, 加锁时会使用读屏障, 强制从主存重新读数据, 也就是说它能保证读的数据是最新的, 然后monitorexit就是解锁, 解锁时通过写屏障保证, 强制将线程本地内存中的变量刷新到主存中, 能够确保线程修改数据后对其

他线程立即可见. 这就是可见性. 这里所说的线程本地内存可以理解为就是Java对CPU缓存的抽象.
- synchronized能够通过内存屏障防止指令重排. 这就是有序性.

synchronized怎么用?

- 修饰普通方法, 锁的是this(当前对象), 也就是一个对象用一把锁
- 修饰静态方法, 锁的是 类.class 对象, 也就是类的所有对象公用一把锁
- 修饰代码块, 那你括号里写什么, 他就锁什么.

很多人都知道synchronized在jdk1.6有优化, 做了锁升级, 那为什么要做锁升级呢?

- 肯定是因为不做锁升级就慢.

那为啥慢呢, 慢在哪儿呢?

- synchronized说到根上, 它加锁还是需要调用操作系统的底层原语mutex, 又涉及到线程的阻塞与唤醒.
- 而java的线程模型是1对1的, 每一个Java线程直接对应一个操作系统的内核级线程, 每次切换线程都需要操作系统切换到内核态, 开销很大.
- 这也是之前synchronized的问题所在.
- jdk1.6对其进行了优化, 有了锁升级的过程, 从无锁到偏向锁到轻量级锁到重量级锁.

那为啥能这么升级呢?

- 事实上一个系统大多数时候是不存在锁竞争的, 经常就只有一个线程去拿锁.
- 比如说, 一个商城系统24小时运行, 从凌晨1点到早上6点会有几个人买东西? 没几个人, 所以几乎没有竞争.
- 高并发往往发生在一些固定的时间点, 比如周末, 比如中午, 晚上休息时间. 其他时间并发量不大
- 所以绝大多数系统都没啥并发, 即便是高并发系统, 也不是时时刻刻都有高并发, 大多数时间, 其实只有一个或者几个线程在抢锁.
- 为了在低并发时, 降低获取锁的代价, 所以就做了锁升级. 锁升级的过程, 其实也是去应对越来越激烈的线程竞争的过程.

锁升级的过程到底是啥样的? 为什么要有偏向锁, 轻量级锁, 重量级锁呢?

- 最开始是无锁状态
- 当有第一个线程来访问同步代码时, JVM将对象头中Mark Word的锁标志位设置为偏向锁，并将线程 ID 记录到对象头的MarkWord中. 那这个线程进入这个同步代码块就不需要再进行其他同步操作了.
    - 为什么要有偏向锁? 偏向锁考虑的是那种只有一个线程访问同步代码的场景.
- 那什么时候升级到轻量级锁呢? 当第二个线程来抢锁, 就升级为轻量级锁. 第二个线程拿不到锁就采用CAS+自旋的方式不断尝试获取锁.
    - 为什么要有轻量级锁? 轻量级锁考虑的是竞争锁对象的线程不多，而且线程持有锁的时间也不长的情景。因为阻塞线程需要操作系统从用户态转到内核态，代价较大，如果刚刚阻塞不久这个锁就被释放了，那这个代价就有点得不偿失了，因此这个时候就干脆不阻塞这个线程，让它自旋这等待锁释放.

    - 那自旋就比阻塞性能好吗? 自旋是让线程一直循环执行抢锁指令, 线程是一直在运行的, 没有被阻塞. 所以就减少了线程上下文切换的开销, 操作系统就不需要切换到内核态了. 短时间的自旋性能是不错的.
    - 但是长时间的自旋会使CPU一直空转, CPU没办法执行其他任务, 会浪费CPU, 所以这里轻量级锁的自旋是适应性自旋, 自旋时间由上一个线程自旋的时间决定的.
    - 可以看到, 偏向锁只记录线程id, 而轻量级锁只是自旋抢锁, 他们都没有做操作系统级的线程阻塞与切换, 不需要操作系统切换到内核态做操作, 整个过程在用户态即可完成. 所以在较低的锁竞争时, 偏向锁和轻量级锁的设计就提高了性能.
- 那轻量级锁什么时候升级到重量级锁呢? 当第二个线程自旋一定次数后还是没拿到锁, 获取锁失败了. 或者, 当有更多线程来抢锁了. 那么就升级为重量级锁.
    - 重量级锁 加锁就需要调用操作系统的底层原语mutex, 所以每次切换线程都需要操作系统切换到内核态, 开销很大. 这也就是为什么称它为重量级锁.
    - 为什么非得升级到重量级锁呢? 因为自旋只适合于锁竞争小, 而且任务执行时间较短的场景. 当有很多线程来抢锁, 你让大量线程自旋, 很浪费CPU. 当某个任务执行时间很长, 那自旋时间就会很长, 长时间自旋也是很浪费CPU的. 所以当第二个线程自旋一定次数后还是没拿到锁, 或者有更多线程来抢锁, 就得直接进入重量级锁.
- 当升级到重量级锁时，对象头Mark Word中的指针会指向锁监视器Monitor.

那为什么要有锁监视器?

- 锁监视器主要用来负责记录锁的拥有者, 记录锁的重入次数, 负责线程的阻塞, 唤醒.

- 准确来说锁监视器就是一个对象, 它有这个几个字段, 首先是owner, 用来保存持有锁的线程. 然后是重入次数的计数器, 用来记录锁的重入次数.
- 然后还有锁池, 和等待池. 锁池主要用来管理抢锁失败的线程. 等待池主要用来管理调用 wait() 方法陷入等待状态的线程.

```
class ObjectMonitor {
  void*      _owner;        // 持有锁的线程
  WaitSet    _WaitSet;      // 等待池（管理调用 wait() 方法的线程）
  EntryList _EntryList;     // 锁池（管理因竞争锁失败而阻塞的线程）
  int _recursions           // 记录锁的重入次数
  // 其他不重要的字段 ...
};
```

owner这个就是记录拿到锁的线程, 不用详细介绍.

那重入次数计数器是干嘛的呢?

- 当一个线程重复的去获取这个锁, 这就是可重入锁.
- 那synchronized可重入怎么实现的? 就是依赖锁监视器中的这个重入次数计数器, 重入一次计数器加1次, 释放一次, 计数器减一次, 减到0就完全释放锁了.

那锁池和等待池是干嘛的呢?

- 在重量级锁状态, 当有线程拿到锁, 此时监视器中的owner字段就记录了拿到锁的线程, 没有拿到锁的线程就被阻塞住, 进入blocking状态, 被放到锁池中.
- 当拿到锁的线程调用了wait()方法, 那该线程就会释放锁, 进入waiting状态, 被放到等待池中.

- 当某个线程调用了notify唤醒这个waiting线程, 那这个线程就从waiting状态变为blocking状态, 然后被放入锁池中, 等待锁释放重新去抢锁.

锁池和等待池? 锁竞争失败的线程和调用了wait()方法的线程有什么区别吗, 为什么放到两个不同的集合中?

- 锁池中放的是竞争锁失败的线程, 线程状态是blocking, 竞争锁失败那他的目标就是尽快获取锁去执行任务. 这是锁的互斥问题.
- 等待池放的线程是主动放弃锁, 等待被唤醒后去配合其他线程完成任务的, 线程状态是waiting或者time_waiting. 这些waiting状态的线程, 是想等某个资源到位了, 等某个事情完成了, 然后再被notify()唤醒, 然后再放入锁池准备抢锁, 做业务. 这是线程通信问题.
- 所以等待池的线程和锁池中的线程目标和要解决的问题是完全是不一样的, 当然要放到两个不同的集合了.

最后梳理一下synchronized的过程

- 最开始是无锁状态
- 当有第一个线程来访问同步代码时, JVM将对象头中的**Mark Word**设置为**偏向模式**，并记录线程ID.
- 当第二个线程来抢锁, 就升级为轻量级锁. 第二个线程拿不到锁就自旋几次.
- 当第二个线程自旋一定次数后还是没拿到锁, 抢锁失败. 或者说, 有更多线程来抢锁了. 那么就升级为重量级锁.
- 但当升级到重量级锁时，Mark Word中的指针会指向Monitor对象。Monitor锁监视器就负责管理线程的阻塞和唤醒.

- 此时监视器中的owner字段就记录了拿到锁的线程, 没有拿到锁的线程就被放到锁池中, 线程状态为blocking.
- 当拿到锁的线程调用了wait()方法, 那该线程就会释放锁, 进入waiting状态, 被放到等待池中.
- 当某个线程调用了notify唤醒这个waiting线程, 那这个线程就从waiting状态变为blocking状态, 然后被放入锁池中, 等待锁释放重新抢锁.